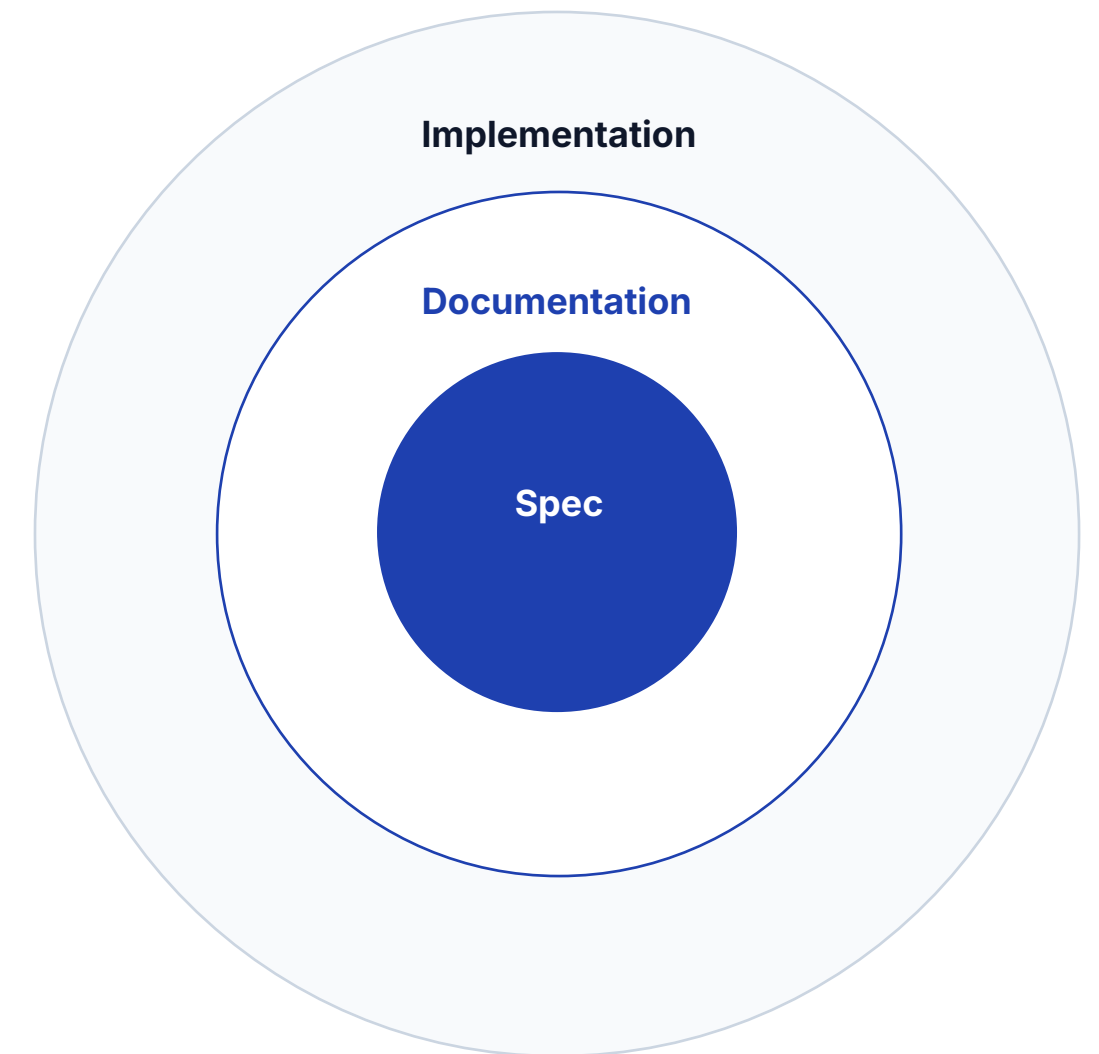


API Specification và OpenAPI

Viết đặc tả API bằng OpenAPI 3.x, sinh tài liệu
và kiểm thử tự động với Swagger UI.



Mục tiêu & chuẩn đầu ra

LLO 4.1

Viết OpenAPI specification cho một REST API đơn giản

Mô tả endpoints, parameters, request body và responses bằng OpenAPI 3.x ở định dạng YAML hoặc JSON.

Đánh giá: **Bài tập thực hành** Liên quan: **CLO3**

LLO 4.2

Tạo API documentation hoàn chỉnh với Swagger UI

Sinh docs tự động từ OpenAPI, cấu hình Swagger UI, đồng bộ spec với implementation qua Flask.

Đánh giá: **Demo + bài tập** Liên quan: **CLO3**

Sau buổi học, sinh viên có thể:

01 Phân biệt

Phân biệt rõ OpenAPI Specification (chuẩn) và Swagger (công cụ); nắm được phiên bản 3.0 vs 3.1.

02 Đọc & viết

Đọc hiểu một OpenAPI document hiện có và viết mới cho API từ 3-5 endpoints.

03 Tích hợp Flask

Dùng `apispec` hoặc `flask-smorest` để render Swagger UI cho Flask app.

Nội dung chính

PART 01



Tại sao cần API Specification?

Vấn đề thực tế khi documentation lạc hậu so với code; sự khác biệt giữa Spec, Document và Implementation; lịch sử Swagger → OpenAPI.

PART 02



Giải phẫu một OpenAPI document

`openapi` · `info` · `paths` · `components` · `security`; mối liên hệ giữa YAML và JSON Schema.

PART 03



Thực hành viết & kiểm tra spec

Viết OpenAPI cho Tasks API (5 endpoints), validate với Swagger Editor, xuất bản spec versioned lên Git.

PART 04



Hệ sinh thái công cụ & tích hợp Flask

Swagger UI vs Redoc vs Stoplight Elements; minh họa Flask + apispec render spec và build Swagger UI cho API đang chạy.

01

Tại sao cần API Specification?

Vấn đề thực tế khi đặc tả không tồn tại; sự khác biệt với documentation và implementation; lịch sử hình thành OpenAPI.

VẤN ĐỀ THỰC TẾ

Hậu quả khi documentation trễ hơn code

KHÁI NIỆM

Specification ≠ Document ≠ Implementation

BỐI CẢNH LỊCH SỬ

Từ Swagger 2010 đến OpenAPI 3.1 (2021)

Một API không có spec thì có vấn đề gì không?

Hệ quả thường thấy

01 Documentation trễ hơn code

Developer A viết endpoint `POST /tasks/:id/assign`; ba tháng sau người mới onboard đọc file Word thấy vẫn ghi `PUT /tasks/:id/assignee` và implement lịch.

02 Tranh cãi giữa frontend & backend

FE team gọi `user.name` trong khi BE trả `full_name`; thiếu spec cố định nên sửa tới lui. QA không biết test theo phiên bản nào.

03 Không sinh được client / SDK / test tự động

Cần generate TypeScript client, Java SDK, hoặc Postman collection — tất cả đều đòi một machine-readable description. Không có spec nghĩa là viết tay và sửa mãi.

04 Khó onboard & đánh giá ngoài

Đối tác muốn tích hợp phải đọc code Python; investor due diligence không thẩm định được API maturity; support team trả lời ticket lặp đi lặp lại.

Tình huống: doc cập nhật

Một team ecommerce 8 người, API tính phí vận chuyển

- Spec viết trong Confluence, 14 trang, sửa lần cuối 4 tháng trước.
- BE Rails trả field `fee_cents`; doc vẫn ghi `fee` (USD).
- FE mobile hard-code theo doc — hiển thị sai 1 đơn vị tiền tệ.
- Sau 2 tuần mới phát hiện khi đối soát; lỗ 31 triệu VND phí ship.
- Mất thêm 1 tuần để reverse-engineer spec thực tế từ code.

Timeline

Tháng 0

Spec đăng Confluence

2 người viết trong 1 tuần. Phần còn lại đọc lướt.

Tháng 2

BE đổi `fee_cents`, ai cũng quên update doc

Commit message chỉ "refactor pricing model".

Tháng 4

Đối soát phát hiện sai 31 triệu VND

FE hiển thị x100 đơn vị trên app mobile.

Tháng 5

Build OpenAPI từ code, tạo Swagger UI

Mất 1 sprint "viết lại spec ngược".

Bài học

Nếu spec tồn tại dưới dạng machine-readable và check được trong CI, BE không thể merge code lệch spec mà không bị pipeline chặn. Doc trễ hơn code là triệu chứng, không phải gốc rễ.

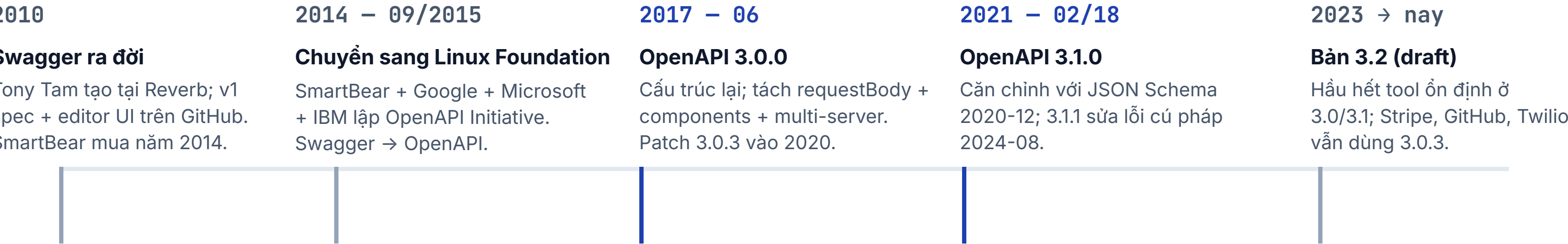
Specification ≠ Document ≠ Implementation

Ba thứ dễ bị gộp vào một; mỗi cái có format, mục đích và vòng đời riêng.

CONTRACT	RENDER	RUNTIME
Specification Định dạng openapi.yaml hoặc openapi.json Đối tượng đọc Công cụ (tool), máy (machine), dev đọc raw Mục đích Mô tả đúng cấu trúc API: paths, schemas, status codes — đủ để máy validate Cập nhật Mỗi commit; check CI lint schema	Documentation Định dạng Trang HTML tĩnh hoặc SPA (Swagger UI, Redoc) Đối tượng đọc Developer tích hợp, partner, người mới Mục đích Sinh tự động từ spec; có ví dụ curl, "Try it out", schema mô tả trường Cập nhật Build mỗi lần spec đổi (CI/CD); không bao giờ sửa tay	Implementation Định dạng Code Flask / Express / FastAPI... Đối tượng đọc Môi trường runtime, OS, scheduler Mục đích Phục vụ request thật; xử lý business logic, validation, persistence Cập nhật Deploy mỗi release; log/metric phát hiện sai lệch với spec

Spec là source of truth · Doc sinh từ Spec · Implementation validate khớp Spec (contract test).

Từ Swagger 2010 đến OpenAPI 3.1 (2021)



OpenAPI ≠ Swagger

- **OpenAPI Specification (OAS)** — chuẩn mô tả API, không phụ thuộc vendor, quản lý bởi OpenAPI Initiative.
- **Swagger** — tập công cụ open-source ban đầu của SmartBear: Editor, UI, Codegen.
- Một số tài liệu cũ vẫn dùng "Swagger" để chỉ spec (Swagger 2.0 ≈ OpenAPI 2.0).

Điểm mấu chốt

OpenAPI là *chuẩn*, Swagger là *bộ tool*. Trong buổi này ta viết spec theo OAS 3.x; render docs bằng Swagger UI; khi nào cần generate SDK / mock server, ta tái sử dụng cùng một file `openapi.yaml`.

02

Phân tích một OpenAPI document

CẤU TRÚC TỔNG THỂ • top-level keys

info / servers / tags • metadata & routing

paths / operations • endpoint anatomy

parameters • requestBody • responses

components/\$ref • security schemes

Cấu trúc một OpenAPI document

Mọi file `openapi.yaml` có một bộ top-level keys theo OAS 3.x; chỉ **openapi** và **info.title** / **info.version** là bắt buộc.

```
FILE OPENAPI 3.1.0
openapi.yaml
openapi: 3.1.0
info:
  title: Tasks API
  version: 2.4.0
  description: Quản lý task nội bộ.
  contact: {email: api@example.vn}
servers:
  - url: https://api.example.vn/v2
  - url: http://localhost:5000/v2
tags:
  - name: tasks # nhóm endpoint
paths:
  /tasks:
    get:
      operationId: listTasks
      responses:
        '200': {$ref: '.../TaskPage'}
components:
  schemas: {Task, Error, ...}
  securitySchemes: {bearerAuth: {...}}
```

Sáu khối top-level

Chỉ hai khối bắt buộc; bốn còn lại tùy chọn.

openapi ★

Phiên bản OAS — bắt buộc. Dùng **3.1.0** cho JSON Schema alignment; **3.0.3** phổ biến.

info ★

title + **version** bắt buộc; có thể thêm **description**, **contact**, **license**.

servers · **tags**

servers cho môi trường (dev/staging/prod); **tags** nhóm endpoint.

paths

Phần quan trọng nhất — URL + HTTP method + parameters + responses.

components

Schema, parameters, responses, securitySchemes — tái sử dụng qua \$ref.

security

Áp dụng bảo mật: apiKey, http (Bearer), oauth2, openIdConnect.

info · servers · tags

Ba khối giúp reader hiểu "API này của ai, chạy ở đâu, nhóm endpoint thế nào" trước khi đọc paths.

1 · Thông tin chung

info — danh thiếp API

- title · version bắt buộc
- description hỗ trợ CommonMark
- contact · license cho compliance
- termsOfService URL

```
info:
  title: Tasks API
  version: 2.4.0
  summary: Quản lý task nội bộ
  contact:
    name: API Team
    email: api@example.vn
  license:
    name: Proprietary
```

2 · Môi trường

servers — base URL

- Mỗi entry: url + description
- Hỗ trợ variables với template {var}
- Swagger UI dùng cho "Try it out"

```
servers:
  - url:
      https://api.example.vn/v2
    description: Production
  - url:
      "https://staging.\
      api.example.vn/v2"
    description: Staging
  - url:
      http://localhost:5000/v2
    description: Local dev
  - url:
      "https://{tenant}.\
      example.vn/v2"
    variables:
      tenant: {default: acme}
```

3 · Phân nhóm

tags — gom endpoint

- Mỗi tag có name + description
- Operation tham chiếu tags: [name]

4 · Link ngoài

externalDocs — link docs dài

```
externalDocs:
  description: Hướng dẫn đầy đủ
  url:
    https://docs.example.vn/api
```

paths và operations

Mỗi path là URL template; mỗi HTTP method dưới path là một operation. Operation khai báo request client gửi và response server trả.

PHÂN TÍCH CÚ PHÁP

Một operation có tới 12 trường — dùng 5–7 là đủ

```
paths:
  /tasks/{taskId}:
    parameters: # path-level
      - name: taskId
        in: path
        required: true
    get:
      tags: [tasks]
      operationId: getTaskById
      responses:
        '200': {description: OK}
        '404': {$ref: '.../NotFound'}
    delete:
      operationId: deleteTask
      responses:
        '204': {description: No content}
```

CÁC METHOD ĐƯỢC HỖ TRỢ

Sáu method theo RFC 9110

GET	Đọc tài nguyên, idempotent, cache được, không có body.
POST	Tạo mới hoặc action không idempotent.
PUT	Replace toàn bộ resource (idempotent).
PATCH	Cập nhật một phần (JSON Merge/JSON Patch).
DELETE	Xoá resource (idempotent).
HEAD	Giống GET nhưng chỉ trả headers.
OPTIONS	Mô tả khả năng; CORS preflight.

Mẹo dùng operationId

operationId phải **unique trong toàn document**. Tool sinh SDK đặt tên method: `api.getTaskById(...)`.

{taskId} path parameter · khai báo trong parameters ở path-level hoặc operation-level.

operationId tool sinh client code đặt tên method theo đây: `api.getTaskById(...)`

Parameters — bốn vị trí

parameter mô tả một input đơn lẻ. OAS 3 cho phép bốn vị trí, mỗi vị trí có validation riêng.

in: path

Nằm trong URL

Định danh resource. Luôn bắt buộc, không null.

Khi nào dùng: viết `/users/{id}`

```
- name: taskId
  in: path
  required: true
  schema:
    type: integer
    format: int64
    minimum: 1
```

in: query

Sau dấu ?

Filter, sort, phân trang. Có thể optional.

Quy tắc: query ≤ 5 trường, không nested.

```
- name: status
  in: query
  schema:
    type: string
    enum: [open, done]
    default: open
- name: limit
  in: query
  schema:
    type: integer
    maximum: 100
```

in: header

HTTP header

Auth token, trace id. Format X-Request-Id.

Chú ý: thường viết Authorization qua securitySchemes.

```
- name: X-Request-Id
  in: header
  schema: {type: string,
            format: uuid}
- name: Authorization
  in: header
  schema: {type: string}
```

in: cookie

Cookie

Phiên làm việc. Ít dùng pure-API.

Lưu ý: OAI 3.1 thêm in: cookie chính thức; 3.0.3 có thể dùng header thay.

```
- name: session
  in: cookie
  required: false
  schema:
    type: string
```

requestBody — payload cho POST/PUT/PATCH

Khác với parameters (key-value đơn lẻ), requestBody cho phép **nhiều content-type** trên cùng operation.

CẤU TRÚC

Ba trường con

description

- Hiển thị trong UI; hỗ trợ CommonMark.

content (bắt buộc)

- Map **media-type** → MediaType Object.

required

- Mặc định false; nên true cho POST/PUT/PATCH.

MediaType Object

- schema · example · examples · encoding

CONTENT NEGOTIATION

Một operation có thể nhận **cả JSON lẫn**

form-urlencoded: khai báo nhiều entry trong **content**. Client chọn qua header **Content-Type**.

JSON vs form-urlencoded

Form-encoded khi browser form submit — nặng nề hơn, gặp khó với nested object. Ưu tiên JSON cho API hiện đại trừ khi client yêu cầu.

POST /tasks – body + responses

post:

operationId: createTask

requestBody:

required: true

description: Tạo task mới cho user hiện tại

content:

application/json:

schema: {\$ref: '.../CreateTaskInput'}

example:

title: "Họp nhóm API team"

dueDate: "2026-09-15"

priority: "high"

responses:

'201':

description: Tạo thành công

headers:

Location: {schema: {type: string}}

content:

application/json:

schema: {\$ref: '.../Task'}

'400': {\$ref: '.../BadRequest'}

'422': {\$ref: '.../ValidationError'}

Mỗi operation khai báo response cho từng status code

Status code là hợp đồng giữa server và client. Theo RFC 9110 thuộc năm nhóm; mỗi nhóm mang một lớp trách nhiệm.

2xx SUCCESS	200 OK: thành công, có body · 201 Created: tạo mới, có header Location · 204 No Content: xóa / action không body
3xx REDIRECT	301 Moved Permanently: đổi URL · 304 Not Modified: dùng cache (có ETag) · 307 Temporary Redirect: giữ method, OAuth callback
4xx CLIENT	400 invalid · 401 missing auth · 403 forbidden · 404 not found · 409 state conflict · 422 validation fail · 429 rate limit
5xx SERVER	500 Internal Server Error: không xác định · 503 Service Unavailable: đang bảo trì / quá tải · 504 Gateway Timeout: upstream timeout

Ví dụ responses block

```
responses:
  '200':
    description: Danh sách task
    content:
      application/json:
        schema: {type: array, items: {$ref: '.../Task'}}
  '400': {$ref: '.../BadRequest'}
  '401': {$ref: '.../Unauthorized'}
  'default': # bắt code không khai báo
    description: Lỗi không mong đợi
    content:
      application/problem+json: {schema: {$ref: '.../Error'}}
```

RFC 7807 — Problem Details

Trả lỗi theo application/problem+json.

```
{
  "type": ".../errors/task-not-found",
  "title": "Task not found",
  "status": 404,
  "detail": "task 42 thuộc user khác",
  "instance": "/v2/tasks/42"
}
```

Tái sử dụng schema Task qua \$ref

Schema Task có thể xuất hiện trong nhiều operation. Copy-paste khiến ba chỗ dễ lệch — \$ref đảm bảo một nguồn.

Ví dụ

`components`

```
components:
  schemas:
    Task:
      type: object
      required: [id, title]
      properties:
        id: {type: string, format: uuid}
        title: {type: string, minLength: 1}
        status: {type: string, enum: [open, done]}
        priority:
          type: string
          enum: [low, normal, high]
        dueDate: {type: string, format: date}
    CreateTaskInput:
      allOf:
        - $ref: '.../Task'
        - type: object
          properties: {assigneeId: {type: string}}
```

Tám nhóm con trong components

Mỗi nhóm là "thư viện" có thể tái tham chiếu.

schemas JSON Schemas · dùng nhiều nhất	responses Response nguyên mẫu
parameters vd: PageSize chung	examples Ví dụ đặt tên để tái dùng
requestBodies Body nguyên mẫu, ít dùng	headers vd: X-RateLimit
securitySchemes 4 kiểu — xem slide trước	callbacks / pathItems Webhook & shared path

Cú pháp \$ref trong OAS

- **Internal:**`$ref: '#/components/schemas/Task'`
- **External file:**`$ref: '../schemas/task.yaml#/Task'`
- **Remote URL:**`$ref: 'https://.../openapi.yaml#/components/schemas/Error'`
- OAS 3.1 hỗ trợ `$id` & `$anchor` (chuẩn JSON Schema 2020-12).

Tái sử dụng schema lỗi và response qua \$ref

Schema Task có thể xuất hiện trong nhiều operation. Copy-paste khiến ba chỗ dễ lệch — \$ref đảm bảo một nguồn.

Ví dụ

components

components:

schemas:

Error: # RFC 7807 problem details

required: [status, title]

properties:

type: {type: string, format: uri}

title: {type: string}

status: {type: integer}

detail: {type: string}

traceId: {type: string}

responses:

NotFound:

description: Không tìm thấy resource

content:

application/problem+json:

schema: {\$ref: '.../Error'}

Tám nhóm con trong components

Mỗi nhóm là "thư viện" có thể tái tham chiếu.

schemas

JSON Schemas · dùng nhiều nhất

responses

Response nguyên mẫu

parameters

vd: PageSize chung

examples

Ví dụ đặt tên để tái dùng

requestBodies

Body nguyên mẫu, ít dùng

headers

vd: X-RateLimit

securitySchemes

4 kiểu — xem slide trước

callbacks / pathItems

Webhook & shared path

Cú pháp \$ref trong OAS

- **Internal:** \$ref: '#/components/schemas/Task'
- **External file:** \$ref: '../schemas/task.yaml#/Task'
- **Remote URL:** \$ref: 'https://.../openapi.yaml#/components/schemas/Error'
- OAS 3.1 hỗ trợ \$id & \$anchor (chuẩn JSON Schema 2020-12).

Security schemes — API Key và HTTP

OAS 3.1 có 4 kiểu: apiKey, http, oauth2, openIdConnect.

type: apiKey

API Key

Token tĩnh truyền qua header / query / cookie. Phổ biến cho API nội bộ.

```
apiKeyAuth:
  type: apiKey
  in: header
  name: X-API-Key
```

type: http

HTTP Authentication

Scheme HTTP chuẩn: Basic hoặc Bearer (≈ JWT).

```
bearerAuth:
  type: http
  scheme: bearer
  bearerFormat: JWT
basicAuth: # admin
  type: http
  scheme: basic
```

ÁP DỤNG security ở hai mức

```
# Toàn cục — mọi operation yêu cầu bearer
security: [{bearerAuth: []}]
# Riêng operation — POST /auth/login bỏ qua auth
paths:
  /auth/login:
    post: {security: []} # mảng rỗng = không yêu cầu
```

Cảnh báo thường gặp

- Khai báo securitySchemes rồi không tham chiếu → Swagger UI không hiện nút Authorize.
- [] (mảng rỗng) = **"không cần auth"**.
- OAuth2 không tự cấp token — FE vẫn cần Authorization Code + PKCE flow.

Security schemes — OAuth 2.0 và OpenID Connect

OAS 3.1 có 4 kiểu: apiKey, http, oauth2, openIdConnect.

type: oauth2

OAuth 2.0

User ủy quyền ứng dụng bên thứ ba — PKCE flow là chuẩn 2026.

```
oauth2:
  type: oauth2
  flows:
    authorizationCode:
      authorizationUrl:
        https://auth.ex.vn/oauth/authorize
      tokenUrl:
        https://auth.ex.vn/oauth/token
    scopes: {tasks:read: Xem, tasks:write: Sửa}
```

type: openIdConnect

OpenID Connect

Đã có identity provider (Google / Auth0 / Keycloak) theo chuẩn OIDC.

```
oidc:
  type: openIdConnect
  openIdConnectUrl:
    "https://auth.ex.vn/.well-known/\
openid-configuration"
```

ÁP DỤNG security ở hai mức

```
# Toàn cục — mọi operation yêu cầu bearer
security: [{bearerAuth: []}]
# Riêng operation — POST /auth/login bỏ qua auth
paths:
  /auth/login:
    post: {security: []} # mảng rỗng = không yêu cầu
```

Cảnh báo thường gặp

- Khai báo securitySchemes rồi không tham chiếu → Swagger UI không hiện nút Authorize.
- [] (mảng rỗng) = **"không cần auth"**.
- OAuth2 không tự cấp token — FE vẫn cần Authorization Code + PKCE flow.

03

Phần ba · Thực hành

Đọc — viết — kiểm tra spec

35 phút cho ba hoạt động: phân tích Petstore (10'), viết Tasks API 5 endpoints (20'), validate bằng Swagger Editor (5').

Công cụ dùng trong phần này

21 · ĐỌC HIỂU

swagger.io/docs — Petstore 3.0 / 3.1 official

22 · VIẾT MỚI

5 endpoints Tasks API (list, get, create, patch, delete)

24 · KIỂM TRA

editor.swagger.io + Spectral CLI lint

BÀI NỘP

Nộp `tasks-api.yaml` qua LMS; đi kèm screenshot
Swagger UI hiển thị được.

Phân tích Petstore — ví dụ chính thức OpenAPI Initiative

Petstore là ví dụ "Hello World" của OpenAPI Initiative — repo `0AI/0penAPI-Specification` có `examples/v3.0/petstore.yaml`.

ĐỌC TRÊN REPO CHÍNH THỨC

petstore.yaml

openapi: 3.0.3

info: {title: Pet Store, version: 1.0.0}

paths:

 /pets:

 get:

 operationId: listPets

 tags: [pets]

 parameters:

 - {name: limit, in: query, schema: {type: integer}}

 responses:

 '200':

 content:

 application/json: {schema: {\$ref: '#/.../Pets'}}

 post:

 operationId: createPets

 requestBody:

 required: true

 content: {application/json: {\$ref: '#/.../Pet'}}

 responses: {'201': {description: Null}}

 /pets/{petId}: # showPetById, deletePet tương tự

components:

 schemas:

 Pet: # bị thiếu error responses

 required: [id, name]

 properties: {id: {type: integer}, name: {type: string},

 tag: {type: string}}

Năm điểm đáng học

① operationId unique

listPets, createPets... Tool sinh SDK đặt method theo đây.

② Tái dùng schema

GET trả Pets = wrapper array của Pet.

③ Thiếu response lỗi

Petstore **không** khai báo 4xx/5xx — bản thực tế phải có default + các mã lỗi riêng.

④ Tag nhóm endpoint

Petstore chỉ một tag pets; project lớn nên có Users, Orders.

⑤ Không có security

Production phải có securitySchemes kể cả khi demo.

Bài tập 1: Tasks API — resource và components

ĐỀ BÀI

Quản lý task nội bộ — title, status, priority, dueDate, assigneeld.

GET /tasks

POST

GET /:id

PATCH

DELETE

FILE OPENAPI 3.0.3 → tasks-api.yaml

```
paths:
  /tasks/{taskId}:
    parameters:
      - in: path
        name: taskId
        schema: {type: string, format: uuid}
    get: {$op: getTaskById}
    patch: {$op: patchTask} # body: UpdateTaskInput
    delete: {$op: deleteTask} # 204 no body
components: # định nghĩa 4 schema + bearerAuth
securitySchemes:
  bearerAuth:
    type: http
    scheme: bearer
    bearerFormat: JWT
```

Validate spec — ba công cụ miễn phí

Sau khi viết, kiểm tra ở ba mức: cú pháp (syntax), chuẩn (spec compliance), best practice (lint).

MỨC 1 – SYNTAX

Swagger Editor

Trình duyệt: `editor.swagger.io`. Validate OAS 3.0/3.1 mỗi lần gõ.

```
# Mở browser, paste YAML
$ npm i -g \
  @swagger-api/swagger-editor
$ swagger-editor start
# Mở port 8080
```

Bắt: schema sai kiểu, \$ref không tồn tại, key viết hoa/thường.

Không bắt: naming chưa tốt, thiếu example.

MỨC 2 – LINT

Spectral CLI

Lint tự động theo ruleset Stoplight. Tích hợp pre-commit và GitHub Actions.

```
# Cài package
$ npm i -g \
  @stoplight/spectral-cli
# Lint theo ruleset
$ spectral lint tasks-api.yaml
# Custom ruleset
$ spectral lint tasks-api.yaml \
  --ruleset .spectral.yaml
```

Rule quan trọng: operation-description, info-contact, oas3-api-servers.

MỨC 3 – CHẤT LƯỢNG

Redocly CLI

Bundle nhiều file + lint nâng cao + preview docs. Thay thế swagger-cli.

```
# Cài
$ npm i -g @redocly/cli
# Lint recommended ruleset
$ redocly lint tasks-api.yaml
# Preview docs
$ redocly preview-docs \
  tasks-api.yaml
```

Tốt: bundle specs ra file đơn, sinh HTML docs ReDoc.

VÍ DỤ OUTPUT SPECTRAL LINT

```
warning  info-contact @ 8:5  Object nên có "contact"
error    oas3-api-servers @ 11:3  Server URL nên HTTPS
```

04

Phần bốn · Công cụ

Hệ sinh thái & tích hợp Flask

SO SÁNH TOOLS

Swagger UI vs Redoc vs Stoplight Elements

FLASK + APISPEC

Sinh OpenAPI từ Flask views, render Swagger UI

FLASK-SMOREST

Code-first OAS 3.x với marshmallow + Swagger UI

Bài tập về nhà

Tích hợp Swagger UI vào Flask app có sẵn; thêm GET /openapi.json trả spec và nhúng Swagger UI vào /docs.

Swagger UI · Redoc · Stoplight Elements — chọn cái nào?

Cả ba đều nhận cùng `openapi.yaml` và sinh HTML. Chọn theo tính năng "Try it out", branding, và load-time.

UI Swagger UI

Chuẩn ngành từ 2013. Có sẵn nút **"Try it out"** gọi API thật. Phổ biến nhất trong các nội bộ và developer portal.

Ưu điểm

- Try it out call API thật
- OAuth2 PKCE flow tích hợp sẵn
- Bundle nhẹ (dist/ ~800 KB)
- flask-smorest, flasgger tích hợp sẵn

Hạn chế

- Layout cũ, khó custom CSS branding
- Sidebar nav kém với spec > 50 paths

Dùng khi: cần Try it out cho QA / đối tác; tích hợp nhanh vào Flask.

RE Redoc

Layout ba cột clean: navigation / spec / code samples. Hướng đến developer-facing docs. Sinh HTML tĩnh cho publish lên GitHub Pages.

Ưu điểm

- Layout đẹp, hỗ trợ right-panel code sample
- Server-side render → SEO, first paint nhanh
- redocly build-docs sinh file HTML tĩnh

Hạn chế

- Không có Try it out (bản Community)
- Tính năng nâng cao cần Redocly Cloud trả phí

Dùng khi: publish docs cho khách hàng ngoài; cần đẹp, branding nhất quán.

EL Stoplight Elements

Web Component — nhúng vào bất kỳ framework (React, Vue, plain HTML) bằng một tag. Có cả Try it out và sidebar nav tốt.

Ưu điểm

- Nhúng một dòng: `<elements-api>`
- Sidebar nav hoạt động tốt với spec lớn
- Có Try it out CORS proxy tích hợp

Hạn chế

- Bundle lớn hơn (~1.2 MB gzip)
- Tài liệu config ít hơn Swagger UI

Dùng khi: nhúng docs vào portal React / Vue; dự án cần sidebar nav linh hoạt.

Flask + apispec — schema và route

apispec đọc YAML docstring + marshmallow Schema → OpenAPI doc.

FILE — app.py (4 bước)

```
# Schema marshmallow → JSON Schema
class TaskSchema(Schema):
    id = fields.UUID(dump_only=True)
    title = fields.Str(required=True)
    status = fields.Str(load_default="open")
# Route + YAML docstring định nghĩa spec
@app.get("/tasks")
def list_tasks():
    """get:
    tags: [tasks]
    responses:
        200:
            content:
                application/json:
                    schema: TaskList
    """
    return jsonify([]), 200
```

Khi nào chọn công cụ nào

apispec phù hợp khi đã có code và muốn ghép YAML docstring thành spec; **flask-smorest** tốt cho *project mới* cần spec-first.

\$ python app.py → http://localhost:5000/docs (Swagger UI) + /openapi.json

Bốn bước luồng

1 Schema → JSON Schema

MarshmallowPlugin dịch sang OAS 3.x.

2 Docstring → paths

FlaskPlugin parse YAML docstring.

3 /openapi.json trả spec

spec.to_dict() trả dict.

4 /docs render UI

CDN Swagger UI.

Nhược điểm

- YAML docstring dễ typo.
- Schema không đồng bộ hai chiều với code.
- → flask-smorest.

Flask + apispec — đăng ký và xuất spec

apispec đọc YAML docstring + marshmallow Schema → OpenAPI doc.

FILE — app.py (4 bước)

```
# Spec instance + đăng ký view
from apispec import APISpec
ap = APISpec(title="Tasks API", version="1.0.0")
with app.test_request_context():
    ap.path(view=list_tasks)

# Serve spec JSON + Swagger UI
@app.get("/openapi.json")
def spec_json():
    return jsonify(ap.to_dict())

@app.get("/docs")
def docs():
    return SWAGGER_HTML # CDN, trả /openapi.json
```

Khi nào chọn công cụ nào

apispec phù hợp khi đã có code và muốn ghép YAML docstring thành spec; **flask-smorest** tốt cho project mới cần spec-first.

\$ python app.py → http://localhost:5000/docs (Swagger UI) + /openapi.json

Bốn bước luồng

1 Schema → JSON Schema

MarshmallowPlugin dịch sang OAS 3.x.

2 Docstring → paths

FlaskPlugin parse YAML docstring.

3 /openapi.json trả spec

spec.to_dict() trả dict.

4 /docs render UI

CDN Swagger UI.

Nhược điểm

- YAML docstring dễ typo.
- Schema không đồng bộ hai chiều với code.
- → flask-smorest.

flask-smorest – cấu hình và schema

flask-smorest sinh spec từ Blueprint.arguments + responses; Swagger UI nhúng sẵn tại /docs.

```
app.py — flask-smorest
pip install flask-smorest

from flask import Flask
from flask_smorest import Api, Blueprint
from marshmallow import Schema, fields
app = Flask(__name__)
app.config["API_TITLE"] = "Tasks API"
app.config["API_VERSION"] = "v1"
app.config["OPENAPI_VERSION"] = "3.0.3"
app.config["OPENAPI_SWAGGER_UI_PATH"] = "/docs"
app.config["API_SPEC_OPTIONS"] = {
    "security": [{"bearerAuth": []}]
}
api = Api(app)
class TaskSchema(Schema):
    id = fields.UUID(dump_only=True)
    title = fields.Str()
    status = fields.Str(dump_default="open")
class CreateTaskInput(Schema):
    title = fields.Str(required=True)
    priority = fields.Str(load_default="normal")
```

So với apispec thuần

TIÊU CHÍ	APISPEC	SMOREST
Syntax check	Không	Có
\$ref auto	Thủ công	Tự động
Pagination	Không	Helper
UI tích hợp	Tự setup	Sẵn

Lựa chọn khác

- **connexion** spec-first
- **flasgger** docstring cũ
- **APIFlask** fork APIFairy

```
CHẠY THỬ
$ python app.py
# /docs Swagger UI
# /openapi.json spec
```

Khi nào chọn gì? **apispec** cho project có sẵn code; **flask-smorest** cho project mới spec-first.

flask-smorest – Blueprint và handlers

flask-smorest sinh spec từ Blueprint.arguments + responses; Swagger UI nhúng sẵn tại /docs.

```
app.py — flask-smorest
pip install flask-smorest

bp = Blueprint(
    "tasks", __name__, url_prefix="/v1/tasks"
)

class TaskCollection:
    @bp.arguments(CreateTaskInput)
    @bp.response(201, TaskSchema)
    def post(self, d):
        return {"id": "new", **d}, 201

    @bp.arguments(Schema, location="query")
    @bp.response(200, TaskSchema(many=True))
    def get(self):
        return [], 200

api.register_blueprint(bp)
```

So với apispec thuần

TIÊU CHÍ	APISPEC	SMOREST
Syntax check	Không	Có
\$ref auto	Thủ công	Tự động
Pagination	Không	Helper
UI tích hợp	Tự setup	Sẵn

Lựa chọn khác

- **connexion** spec-first
- **flasgger** docstring cũ
- **APIFlask** fork APIFairy

```
CHẠY THỬ
$ python app.py
# /docs Swagger UI
# /openapi.json spec
```

Khi nào chọn gì? apispec cho project có sẵn code; **flask-smorest** cho project mới spec-first.

Mười best practices khi viết OpenAPI

01 operationId là duy nhất và ổn định

Đặt `createTask`, `getTaskById`; **không đổi tên** sau khi đã ship vì SDK client sẽ phải refactor. Một khi đã phát hành, treat như API contract.

02 Example cho mỗi schema, không chỉ type

`type: string` là chưa đủ. Cung cấp `example` cho mỗi field; Swagger UI hiển thị sample payload giúp FE dev hiểu nhanh.

03 Đặt tên response lỗi ngay từ đầu

Khai báo `Unauthorized`, `NotFound`, `ValidationError` trong `components.responses`. Tái sử dụng qua `$ref` ở mọi operation.

04 Dùng tags để gom nhóm chức năng

Một API 50 endpoints thì **không** thuộc một tag default. Tách thành `Users`, `Orders`, `Billing` cho dễ navigate.

05 Đánh dấu deprecated fields qua extension

Dùng `x-deprecated: true` hoặc `x-sunset` để cảnh báo sớm. Production gateway có thể từ chối sớm trước khi xoá.

06 description có ý nghĩa, không phải lời giải thích

"Returns the list" thừa. Thay bằng hướng dẫn cụ thể: *"page 1-based, tối đa 100 items, sort theo created_at desc"*.

07 Mô tả validation rõ ràng

`minLength`, `maximum`, `enum`, `pattern` — viết hết vào schema. Tool generator sinh client có validation ngay từ spec.

08 Lint spec trong CI

Chạy `spectral lint` trong GitHub Action; merge bị chặn nếu error. Comment warning trong PR.

09 Version spec theo ngày, không "v1"

Giữ `info.version: 2026-08-26` cho pace nhanh.

10 Commit spec cùng code, REVIEW cùng PR

PR đổi endpoint kèm `openapi.yaml`; reviewers từ chối nếu thiếu spec.

Tám sai lầm thường thấy khi viết OpenAPI

Các lỗi phổ biến từ review spec thật; mỗi lỗi kèm bản sửa cụ thể.

LỖI #1Đặt 200 cho mọi response, kể cả lỗi ✗ SAI <pre>responses: '200': description: "Có thể là error"</pre> ✓ ĐÚNG <pre>responses: '200': ok '404': NotFound '422': ValidationError 'default': ServerError</pre>	LỖI #2Không có example FE dev đọc schema vẫn phải tự nghĩ giá trị; <code>example</code> giúp Swagger UI sinh payload mẫu. <pre>properties: status: {type: string, enum: [open, done], example: "open"} # ← thêm dòng này</pre>
LỖI #3Inconsistent naming Endpoint <code>/users</code> chỗ này, <code>/user</code> chỗ khác. Field <code>userId</code> VS <code>user_id</code>. Dùng Spectral rule <code>naming-convention</code>.	LỖI #4Quên security trên một số endpoint Khai báo <code>security</code> global nhưng quên apply cho endpoint lồng nhau. Audit tự động: đếm operation không có security trong CI.
LỖI #5\$ref tới file chưa bundle <code>\$ref: ./schemas/task.yaml</code> — tool có thể không fetch được. Dùng Redocly CLI bundle hoặc copy mọi \$ref vào một file trước khi phát hành.	LỖI #6Trộn vendor extension x- vào schema chính <code>x-google-backend</code>, <code>x-codegen-request-name</code> → khó tái sử dụng ở hệ thống khác. Đặt vào <code>components.x-*</code> hoặc extension file riêng.
LỖI #7Date không chuẩn ISO 8601 Một số team ghi <code>"13/09/2026"</code>. OpenAPI yêu cầu RFC 3339: <code>2026-09-13</code> cho date.	LỖI #8Boolean field tên mơ hồ <code>flag</code>, <code>is_done</code> khó hiểu. Đặt <code>is_archived</code>; prefix <code>is_/has_</code> rõ nghĩa.

Tổng kết buổi học & chuẩn bị Buổi 5

PHẦN 1 · ĐÃ HỌC

Vấn đề & định nghĩa

Spec giải quyết doc-trễ-code; phân biệt Spec ≠ Document ≠ Implementation; lịch sử OpenAPI 3.x.

PHẦN 2 · ĐÃ HỌC

Giải phẫu OpenAPI

openapi · components · security.

PHẦN 3 · ĐÃ LÀM

Viết & kiểm tra spec

Petstore, Tasks API 5 endpoints, validate bằng Editor + Spectral.

PHẦN 4 · ĐÃ THẤY

Hệ sinh thái + Flask

Swagger UI vs Redoc vs Elements; demo Flask + apispec; thấy flask-smorest.

BẠN CÓ THỂ TỰ KIỂM TRA

Nếu chưa làm được hết dưới đây, dành thêm 1-2 giờ đọc OAI/OpenAPI-Specification: đọc hiểu 1 spec 50-100 dòng; viết spec với \$ref; triển khai Flask có /openapi.json + /docs.

Buổi 5 · Data Modeling & Resource Design

Đọc trước — Higginbotham Ch.6

Buổi 5 đi sâu vào **resource modeling**: quan hệ 1-1, 1-n, n-n; pagination strategies (offset vs cursor); filtering & sorting.

Ba câu hỏi thực tế

- Khi nào trả Task đầy đủ, khi nào trả TaskSummary?
- Cursor vs offset pagination — đánh đổi gì?
- Nested resource (/users/{id}/tasks) vs query — khi nào cái nào tốt hơn?

BÀI TẬP VỀ NHÀ · HẠN CHỦ NHẬT 23:59

1. Repo tasks-api/ trên GitHub nhóm
2. File openapi.yaml cho 5 endpoints
3. Flask app có /openapi.json + /docs
4. Screenshot Swagger UI với "Try it out" hoạt động
5. README.md mô tả 2 quyết định thiết kế khó nhất

TÀI LIỆU THAM KHẢO

- Higginbotham, Principles of Web API Design · ch.7 · • Geewax, API Design Patterns · ch.2 · • Pedro, Building an API Product · ch.4
- spec.openapis.org/oas/v3.1.0 · swagger.io/docs · flask-smorest.readthedocs.io